

An Efficient Construction of Completely Independent Spanning Trees in Dense Gaussian Networks

Zaid Hussain^{1*†}, Fawaz AlAzemi^{1†} and Bader AlBdaiwi^{2†}

¹Computer Science Department, Kuwait University, Kuwait.

²Retired, Computer Science Department, Kuwait University, Kuwait.

*Corresponding author(s). E-mail(s): zhussain@cs.ku.edu.kw;

Contributing authors: fawaz@cs.ku.edu.kw; bdaiwiV@cs.ku.edu.kw;

[†]These authors contributed equally to this work.

Abstract

Fault tolerance in routing and broadcasting is a critical aspect in ensuring the reliability and robustness of communication networks, particularly in environments prone to failures. This work presents an efficient method for constructing Completely Independent Spanning Trees (CISTs) within dense Gaussian networks, providing improved fault tolerance, reliability, and communication efficiency in large-scale interconnection systems. To construct the CISTs in the Gaussian network, we partition the network into sets, and accordingly the nodes are connected properly to form the first CIST and then rotated to get the second CIST with less depth than the existing state-of-art. To evaluate the performance of the proposed construction, we calculated the average maximum number of steps required to deliver a message from the root node to all other nodes in the network. A comparison with existing approaches shows that our construction outperforms them, achieving an improvement of at least 33%.

Keywords: Interconnection network, Spanning trees, routing, broadcasting, fault-tolerant.

1 Introduction

With recent advancements in information technology (NLP, LLM, generative AI, etc) and the exponential growth of data, the demand for computation continues to

rise rapidly. Cloud computing has become a pivotal solution to meet this escalating demand. Cloud computing resources are hosted in datacenters, where communication is facilitated by the underlying network infrastructure. Datacenters are designed to be horizontally scalable, allowing for the addition of more computing nodes to distribute the load effectively. As the number of nodes increases, the necessity for efficient and high-performance networks becomes critical. These networks serve as a backbone, and their performance directly impact the overall efficiency and effectiveness of the entire datacenter. Ensuring robust and efficient network infrastructure is paramount to maintain seamless operations and scalability of modern datacenters.

The choice of network topology is crucial for optimizing both the cost and performance of large-scale computing systems due to its direct impact on communication protocols. Various topologies have been proposed in the literature, including Torus [1], Tofu[2], Dragonfly [3], Mesh, Eisenstein-Jacobi (EJ) [4], Gaussian [5], BCube [6], and k -ary n -cube [7], hypercube [8], etc. Each topology is suited to a set of requirements and scenarios and has unique properties in terms of symmetry, regularity, diameter, structure, average hop distance, number of independent spanning trees, fault tolerance levels, etc. For example, the torus topology is well-known and has been employed to connect supercomputers such as IBM BlueGene[9] and Fugaku[10] (ranked No. 1 on June 2020 on Top500 list [11]). Torus networks are symmetric and regular where they provide high performance and economic scalability due to their high path diversity. This enables them to handle high traffic loads without performance degradation even in the presence of a limited number of faulty nodes or links. Similarly, the first Exascale Supercomputer Frontier [12] (ranked No.1 on June 2024 Top500 List [11]) utilizes Dragonfly topology in combination with Slingshot interconnect [13] to improve the overall performance.

The Gaussian network has been proposed in [5][14] as a topology that shares many characteristics with the torus network such as symmetry and regularity. However, the Gaussian network has a smaller diameter and average hop distance when compared to the torus network. Therefore, it offers an overall reduction in network latency and, as a result, a more attractive option. A dense Gaussian network is a specific instance of Gaussian network that offers the maximum number of nodes for a given diameter. Formal definitions and characteristics will be discussed in Section 3.

Independent Spanning Trees (ISTs) are spanning trees rooted at the same node such that, for every other node in the network, the paths from the root to that node in different trees are internally node-disjoint except at the endpoints. A stronger variant, known as Completely Independent Spanning Trees (CISTs), further requires these paths to be both internally node-disjoint and edge-disjoint, with the root node placed at a different location in each tree, providing a higher level of fault tolerance. These structural properties ensure that even if some links or nodes fail, alternative paths still exist, allowing messages to be delivered reliably. In interconnection networks, ISTs and CISTs play a crucial role in fault-tolerant routing and broadcasting, as they provide multiple independent communication paths between nodes. This redundancy improves the resilience of the network by minimizing the impact of failures, reducing congestion, and ensuring reliable message delivery throughout the network. More details on ISTs and CISTs are provided in Section 2, while their definitions and variations are given in Section 3.

In this paper, we present two efficient algorithms (one sequential and one parallel) for constructing CISTs in dense Gaussian networks. The time and communication complexities of the proposed sequential algorithm are, in respective order, $O(n)$ and $O(d)$ where n is the number of the nodes in the network and d is the maximum depth of the CISTs. The time complexity of the parallel algorithm is $O(1)$ and its communication complexity is $O(1)$. For a given diameter k , both algorithms construct CISTs with a lower tree depth of $d = 3k - 1$ when compared to Pai’s algorithm [15], thereby reducing the average distance and the number of steps required to route a message. Additionally, we use CISTs to design a fault-tolerant routing protocol. A theoretical and experimental comparison of our algorithms with Pai’s algorithm is also discussed.

The paper is organized as follows. The literature review is discussed in Section 2. Section 3 provides necessary background and related works. The theoretical foundation for CISTs in dense Gaussian networks is presented in Section 4. Sequential and parallel algorithms are proposed in Section 5, while reliable routing is discussed in Section 6. All experimental results are analyzed in Section 7 followed by the conclusion in Section 8.

2 Literature Review

The Gaussian network was introduced in [5, 14] as a new interconnection topology to improve the performance and scalability of parallel systems [16]. Since their introduction, Gaussian networks have been extensively studied in the literature. For example, [17] presents an efficient algorithm for constructing 1-to-many node-disjoint paths. Additionally, an all-to-all broadcasting algorithm for on-chip interconnects is proposed in [18, 19]. An n -dimensional variant of the Gaussian network is discussed in [20].

ISTs are utilized in many network topologies and are defined as a set of spanning trees in a given network that share the same root node where the paths between the root node and any other node in different trees are node-disjoint except for the two paths’ endpoints. This characteristic of ISTs helps to easily achieve reliability and security in communication protocols [21–24]. Edge-Disjoint Independent Spanning Trees (EDNISTs) are the same as ISTs except that the edges used in one tree are not used in any other trees [25]. CISTs differ from EDNISTs in that each spanning tree has a distinct root node location.

CISTs were first proposed by Hasunuma in [26, 27] and are more preferable than ISTs due to the flexibility for the selection of root nodes for each spanning tree. Similarly to ISTs and EDNISTs, CISTs have a wide range of applications in communication protocols [15, 28–30]. Recently, Pai et al. [15] introduced a sequential algorithm for constructing CISTs in dense Gaussian Networks. The algorithm begins by identifying a Hamiltonian cycle, then each spanning tree utilizes a subset of edges from this cycle along with an additional unique set of edges not included in the cycle. The specific steps of the algorithm vary slightly depending on whether the diameter of the network is odd or even.

ISTs have important applications in reliable communication protocols, broadcasting, and secure message distribution in many topologies, including Gaussian networks

[21], dense Gaussian Networks [25], star networks [24], Parity Cubes [22], and Mobius cubes [23]. However, CISTs offer additional properties compared to ISTs, making them particularly advantageous for various applications. However, finding two or more CISTs in a general graph is an NP-complete problem [27]. Despite this complexity, it is feasible to find CISTs in certain classes of graphs. For instance, [29] presents an algorithm to find the maximum number of CISTs in line graphs. In [31], the authors construct CISTs in BCube Connected Crossbar networks and use them to design fault-tolerant routing protocols. Similarly, CISTs for augmented cube networks are discussed in [30]. The application of CISTs for secure routing protocols in Crossed Cube networks is explored in [32]. Additionally, [28] proposes an ILP formulation to construct CISTs in Ad Hoc networks, which is used to improve overall network performance.

Table 1 presents the differences between ISTs, EDNISTs and CISTs based on previous work and their applications in several interconnection networks.

Table 1 Comparison of ISTs, EDNISTs, and CISTs.

Type	Key Properties	Applications
ISTs	Ensure reliability and security; suitable for fault-tolerant routing and broadcasting.	Used in Gaussian networks [21], dense Gaussian networks [25], star networks [24], parity cubes [22], and Mobius cubes [23].
EDNISTs	Provide edge-disjointness in addition to node-disjointness.	Applied in dense Gaussian networks [25].
CISTs	More flexible due to multiple root selections; NP-complete to find in general graphs; feasible in specific classes of networks.	Algorithms for line graphs [29], BCube Connected Crossbar networks [31], augmented cube networks [30], crossed cubes [32], and Ad Hoc networks [28]; also studied in Gaussian networks [15].

3 Background

Computer network topologies are studied and analyzed using abstract graphs, a tool from graph theory. In this section, we present the essential graph theory background necessary to mathematically and algorithmically represent a Gaussian network topology and to develop a construction algorithm for CISTs.

A graph $G = (V, E)$ is an ordered pair consisting of a set of nodes V and a set of edges E . Nodes represent computation units or network devices while each edge $e = (v, u)$ represents a connection link between an unordered pair of nodes $v, u \in V$. We assume edges in G are undirected and not duplicated. Therefore, G is an undirected and simple graph.

Two nodes $v, u \in V$ are adjacent or neighbors if $(v, u) \in E$. The degree of a node v , denoted as $\deg(v) \geq 0$, represents the total number of neighbors it has. G is n -regular if $\deg(v) = n$ for every node in G . A path P from v_1 to v_n in G is an ordered sequence of nodes $(v_1, v_2, \dots, v_n)$ such that each consecutive pair v_i, v_{i+1} are adjacent (i.e., $(v_i, v_{i+1}) \in E$). The length of the path P is the number of edges it traverses. The shortest path between v, u is a path from v to u with minimum possible length L . L is also denoted as the distance $d(v, u) = L$ between v, u . Two paths P_1, P_2 are edge-disjoint if they share no common edges. Moreover, P_1, P_2 are node-disjoint if they

share no common nodes. A cycle is a path where $v_1 = v_n$. The diameter $D(G)$ equals the maximum length of all shortest paths in G . A graph is connected if there is a path between each pair of its nodes. A tree is a connected graph with no cycles. A spanning tree $ST = (V, E')$ of G is a subgraph of G that contains all nodes in G and $E' \subseteq E$ and it is itself a tree. A set of spanning trees $T_1, T_2, \dots, T_n$ on G forms a set of ISTs if:

1. all trees are edge-disjoint.
2. all trees have the same root node r .
3. the two paths between r and v in any two trees T_i and T_j are node-disjoint except for r and v .

Moreover, Hasunuma introduced the concept of CISTs in [26, 27], which extends ISTs by allowing the spanning tree roots to be distinct in all trees. Formally, a set of spanning trees $T_1, T_2, \dots, T_m$ is a CIST on G if:

1. all trees are edge-disjoint,
2. $r_1, \dots, r_m$ are root nodes for $T_1, T_2, \dots, T_m$, respectively, such that $r_i \neq r_j$ for $1 \leq i, j \leq m$,
3. the two paths between v, u in any two trees T_i, T_j is node-disjoint except for v, u .

The Gaussian network is designed based on the Gaussian integers $\mathbb{Z}[i] = \{x + yi \mid x, y \in \mathbb{Z}\}$. Here, $\mathbb{Z}[i]$ represents a subset of the complex numbers $\mathbb{C}$ where $\mathbb{Z}$ is the set of integers and $i = \sqrt{-1}$. $\mathbb{Z}[i]$ forms a Euclidean domain with the norm $N(\mathbb{Z}[i]) = \|a + bi\| = a^2 + b^2$. For any $\alpha, \beta \in \mathbb{Z}[i]$ with $\alpha \neq 0$ there exist $q, r \in \mathbb{Z}[i]$ such that $\beta = q\alpha + r$. Consequently, $\beta \bmod \alpha = r$ with $\|r\| < \|\alpha\|$. This property of the Gaussian integers underpins the structure and design of Gaussian networks.

Given a Gaussian integer $\alpha = a + bi \neq 0$, a Gaussian network is a graph $G = (V, E)$ where its nodes V are the elements of the residue class modulo α in $\mathbb{Z}[i]$. The Gaussian integer α is referred to as the generator of the graph G . Two nodes $x, y \in V$ are adjacent if $x - y \bmod \alpha \in \{\pm 1, \pm i\}$. Consequently, the Gaussian network G is a 4-regular and symmetric graph. Therefore each node has exactly four neighbors and the graph is invariant under rotations and reflections. The total number of nodes in G corresponds to the norm $\|\alpha\| = a^2 + b^2$. A Gaussian network is considered dense when it contains the maximum number of nodes for a given diameter $k = a = b - 1$ where $\alpha = a + bi$.

4 Completely Independent Spanning Trees

Given a dense Gaussian network $G = (V, E)$ generated by $\alpha = a + bi$ such that $b = a + 1$, where $k = a = b - 1$ is the diameter of the network. The node set V can be divided into 10 subsets, as shown in Fig. 1, and they are represented as follows (R is the root node).

$$\begin{aligned}
R^j &= \{xi^j + yi^{j+1} \mid (x = \frac{k}{2} + 1, y = \frac{k}{2} - 1, k \text{ is even}) \\
&\quad \text{or } (x = \lceil \frac{k}{2} \rceil, y = \lfloor \frac{k}{2} \rfloor, k \text{ is odd})\} \\
S^j &= \{xi^j + yi^{j+1} \mid x = k, y = 0, k > 2\} \\
B1^j &= \{xi^j + yi^{j+1} \mid 0 < x < x_r, y_r < y < k, |x| + |y| = k\} \\
B2^j &= \{xi^j + yi^{j+1} \mid x_r < x < k, 0 < y < y_r, |x| + |y| = k\}
\end{aligned}$$

$$\begin{aligned}
B3^j &= \{xi^j + yi^{j+1} \mid -k < x \leq -y_r, -x_r \leq y < 0, |x| + |y| = k\} \\
B4^j &= \{xi^j + yi^{j+1} \mid -y_r < x \leq 0, -k \leq y < -x_r, |x| + |y| = k\} \\
L1^j &= \{xi^j + yi^{j+1} \mid -k \leq x \leq 0, 0 \leq y \leq k, |x| + |y| = k\} \\
L2^j &= \{xi^j + yi^{j+1} \mid 0 < x < k, -k < y < 0, |x| + |y| = k\} \\
D1^j &= \{xi^j + yi^{j+1} \mid -k < x < k, 0 \leq y < k, |x| + |y| < k\} \\
D2^j &= \{xi^j + yi^{j+1} \mid -k + 1 < x < k - 1, -k < y < 0, |x| + |y| < k\}
\end{aligned}$$

where $j = 0$ or 1 and used to represent, in respective order, the subsets of the first and second CISTs in Gaussian networks. Note that, x_r and y_r are the x and y of R , respectively.

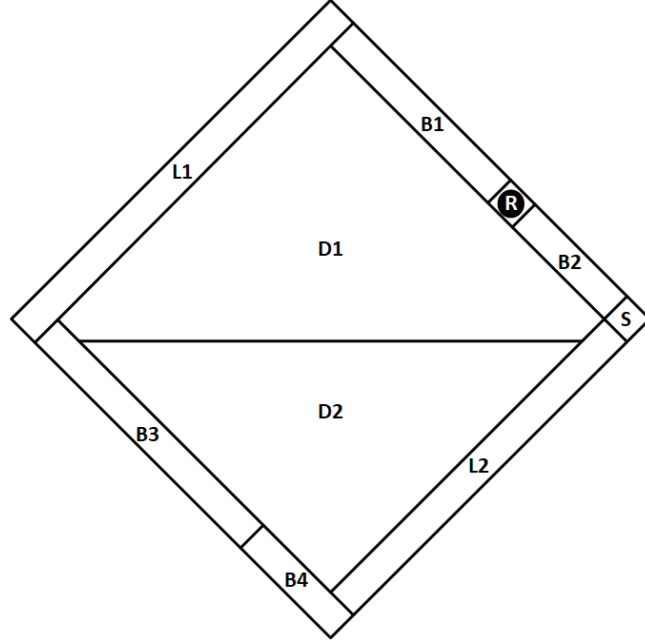


Fig. 1 Node subsets of Gaussian network.

Lemma 1 The subsets contain all nodes in G

Proof Let G be a Gaussian network generated by $\alpha = a + bi$ where $b = a + 1$ and its diameter is $k = a$. When $j = 0$ (i.e., no rotation is applied), let $|A|$ represents the number of nodes in set A . Based on the subsets mentioned above, then $|R^0| = 1$, $|S^0| = 1$, $|B1^0| + |B2^0| = k - 2$, $|B3^0| + |B4^0| = k$, $|L1^0| = k + 1$, $|L2^0| = k - 1$, $|D1^0| + |D2^0| = 2k^2 - 2k + 1$. By summing up all together, we get a total number of nodes equal to $2k^2 + 2k + 1$, which is equal to $N(\alpha)$. Thus, the subsets contains all nodes in G . $\square$

Lemma 2 The subsets are disjoint

Proof For $k > 2$, $j = 0, 1$, and $V = \{R^j\} \cup \{S^j\} \cup \{L1^j\} \cup \{L2^j\} \cup \{B1^j\} \cup \{B2^j\} \cup \{B3^j\} \cup \{B4^j\} \cup \{D1^j\} \cup \{D2^j\}$ represents all nodes in the network. For any two distinct subsets $X, Y \in V$, and $X \neq Y$, we have $X \cap Y = \emptyset$. Thus, all subsets are disjoint. $\square$

Lemma 3 $|CISTs| \leq 2$

Proof Let G be the Gaussian network. $N(\alpha) = 2k^2 + 2k + 1$, and total edges is $4k^2 + 4k + 2$. For $k > 2$, we have exactly $2k^2 + 2k$ edges in each CIST. Thus, we have at most 2 CISTs in G . $\square$

Lemma 4 $CIST_1$ is connected

Proof We prove that every node of $CIST_1$ is reachable from the root node R^0 . Let $v = x + yi$ be any node in the dense Gaussian network. By Lemma 1, v belongs to one of the subsets

$$R^0, S^0, B1^0, B2^0, B3^0, B4^0, L1^0, L2^0, D1^0, D2^0.$$

The construction assigns outgoing edges according to the subset containing the current node. Starting from the root R^0 , the edges $(x, x + 1)$, $(x, x - 1)$, and $(x, x + i)$ connect the root to the three main branches of the construction.

For nodes in $D1^0$ and $D2^0$, the construction uses horizontal edges $(x, x - 1)$ and $(x, x + 1)$, respectively. Hence, every node in these inner subsets is connected along a horizontal chain that eventually reaches either the root branch or one of the boundary subsets. Similarly, nodes in $B1^0$ and $B2^0$ are connected through the edges $(x, x - 1)$, $(x, x + i)$, and $(x, x + 1)$, while nodes in $B3^0$ and $B4^0$ are connected through the edges $(x, x - 1)$, $(x, x + 1)$, and $(x, x - i)$. Therefore, each boundary node is connected to either an adjacent boundary node, an inner node, or a node closer to the root branch.

The subsets $L1^0$ and $L2^0$ have no outgoing edges because they form terminal leaves of the tree. However, each node in $L1^0$ or $L2^0$ has an incoming edge from a neighboring node in one of the adjacent subsets, namely $D1^0$, $D2^0$, $B3^0$, or $B4^0$. Thus, these nodes are also reachable from R^0 .

Therefore, every node of G is reachable from R^0 using the edges defined for $CIST_1$. Since the construction includes all nodes of G and provides a path from the root to every other node, $CIST_1$ is connected. $\square$

Lemma 5 $CIST_2$ is obtained from rotating $CIST_1$

Proof Let ρ denote the rotation mapping on the Gaussian network defined by

$$\rho(x + yi) = i(x + yi) = -y + xi.$$

This mapping corresponds to a counterclockwise rotation by 90° in the Gaussian integer plane. Since multiplication by i preserves adjacency in the Gaussian network, if (u, v) is an edge, then $(\rho(u), \rho(v))$ is also an edge. Therefore, ρ is an automorphism of the Gaussian network.

The first completely independent spanning tree $CIST_1$ is constructed from the subsets

$$R^0, S^0, B1^0, B2^0, B3^0, B4^0, L1^0, L2^0, D1^0, D2^0$$

using the corresponding edge rules for $j = 0$. Applying the rotation ρ to every node in these subsets produces the rotated subsets

$$R^1, S^1, B1^1, B2^1, B3^1, B4^1, L1^1, L2^1, D1^1, D2^1.$$

Similarly, applying ρ to every edge of $CIST_1$ transforms each direction as follows:

$$+1 \mapsto +i, \quad +i \mapsto -1, \quad -1 \mapsto -i, \quad -i \mapsto +1.$$

Thus, every edge used in $CIST_1$ is mapped to the corresponding rotated edge used in the construction of $CIST_2$. More precisely, if

$$E(CIST_1) = \{(u, v) \mid u \in X^0 \text{ and } v \text{ is selected according to the edge rule of } X^0\}$$

where X^0 is one of the subsets of the construction, then the rotated edge set is

$$\rho(E(CIST_1)) = \{(\rho(u), \rho(v)) \mid (u, v) \in E(CIST_1)\}$$

By the definition of the rotated subsets and the rotated directions, this edge set is exactly the edge set assigned to the construction with $j = 1$. Hence,

$$E(CIST_2) = \rho(E(CIST_1)).$$

Since ρ is a graph automorphism, it preserves incidence, adjacency, connectivity, and acyclicity. Therefore, the rotated image of the connected spanning tree $CIST_1$ is also a connected spanning tree. Consequently, applying one rotation to the node subsets and edge rules of $CIST_1$ produces the second completely independent spanning tree $CIST_2$. $\square$

Theorem 6 *The Gaussian network G generated by $\alpha = a + bi$ where $b = a + 1$ and $k = a > 2$ contains two Completely Independent Spanning Trees.*

Proof By Lemma 1, the defined subsets contain all nodes of the dense Gaussian network G , and by Lemma 2, these subsets are pairwise disjoint. Lemma 4 shows that the edge set $E(CIST_1)$ forms a connected spanning tree. Moreover, by Lemma 5, rotating the node and edge sets of $CIST_1$ once produces the second spanning tree $CIST_2$.

It remains to verify the complete independence property. First, $CIST_1$ and $CIST_2$ are edge-disjoint, since the construction of $CIST_2$ is obtained by rotating the directions used in $CIST_1$, and no edge used in $E(CIST_1)$ is reused in $E(CIST_2)$. Hence,

$$E(CIST_1) \cap E(CIST_2) = \phi.$$

Second, the roots of the two spanning trees are distinct because $R^0 \cap R^1 = \phi$.

Now consider any two distinct nodes $u, v \in V(G)$. Since each $CIST_i$ is a tree, there is a unique path between u and v in each tree. In the proposed construction, each non-root node has a unique parent in each tree, as specified by the corresponding parent-child rules, and the parent-child direction assigned to a node in $CIST_2$ is obtained by rotating the direction assigned in $CIST_1$. Therefore, the internal vertices used by the path between u and v in $CIST_1$ belong to a sequence of subsets different from the rotated sequence used by the corresponding path in $CIST_2$. Since the subsets are pairwise disjoint and the two constructions use disjoint parent-child relations, the two paths can share only the endpoints u and v . Thus, the paths between any two nodes in $CIST_1$ and $CIST_2$ are internally node-disjoint.

Consequently, $CIST_1$ and $CIST_2$ are edge-disjoint spanning trees with distinct roots, and the paths between any two nodes in the two trees are internally node-disjoint. Therefore, $CIST_1$ and $CIST_2$ are Completely Independent Spanning Trees in the dense Gaussian network generated by $\alpha = a + bi$, where $b = a + 1$ and $a > 2$. $\square$

Lemma 7 The depth of both CISTs is $3k - 1$.

Proof The $CIST_1$ is used in this proof to prove its depth, and the same argument could be applied to prove the depth of the $CIST_2$.

Notice that the $CIST_1$ is using all the horizontal edges (i.e., ± 1), excluding the wraparound edges. Based on the network subsets explained in the beginning of this section, when k is odd, it is observable that the longest path between two nodes in $CIST_1$ contains (horizontally) the nodes from $-k$ to k with length $2k$. Note that, these connected nodes belongs to $L1 \cup D1 \cup S$. The total wraparound edges in $CIST_1$ is $2k$. Thus, the maximum depth is $4k$ when the root node is located at node ki , which leads to the longest path of length $4k$ from node ki to node $-k$. That is, all the wraparound edges and the edges between the nodes $-k$ and k are used. The depth $4k$ can be reduced into $3k - 1$ by locating the root node approximately in the middle of $B1 \cup R \cup B2 \cup S$, i.e. $R = \lceil k/2 \rceil + \lfloor k/2 \rfloor i$, when k is odd. That is, we use $k - 1$ wraparound edges instead of $2k$. We conclude that the depth of $CIST_1$ is $2k$ (horizontal edges) + $k - 1$ (wraparound edges) = $3k - 1$, which is the length of the longest path from nodes R to $-k$.

On the other hand, when k is even, choosing $R = (k/2) + (k/2)i$ then it leads to have a path of length $3k$ consisting of horizontally connected nodes from $-k$ to k with length $2k$ plus the k wraparound edges. Observe that the second longest path has a length of $3k - 2$. That is, from R we can reach the node $-(k - 1) - i$ using $k - 1$ wraparound edges and the horizontally connected nodes from $-(k - 1) - i$ to $(k - 1) - i$ with length $2k - 2$, which leads the total path length is $3k - 3$. Both path lengths $3k$ and $3k - 3$ can be, in respective order, $3k - 2$ and $3k - 1$ when choosing $R = (k/2 + 1) + (k/2 - 1)i$. Thus, we get the minimum longest path length from node R to $(k - 1) - i$ with length $3k - 1$. $\square$

5 Sequential and Parallel Construction Algorithms

In this section, we present the sequential and the parallel construction algorithms of CISTs in Gaussian networks. Both algorithms are based on the connectivity of CISTs according to the subsets mentioned in Section 4.

5.1 Sequential Construction

To sequentially construct the CISTs, let $j = 0$ (i.e., no rotation). Each receiving node forwards the message to its neighbor(s) using the proper direction(s) (output port(s)) based on its subset belonging to. Table 2 illustrates the required directions to forward the message in order to sequentially construct the first completely independent spanning tree.

Algorithm 1 illustrates the sequential process of forming the first and second Completely Independent Spanning Trees. The algorithm requires three parameters: c as the receiving node which receives the construction packet from the sender, Table 2 to determine which direction (output port) to forward the construction packet based on the location of c , and j either 0 or 1 to know which tree to be constructed.

The sequential construction works as follows. At beginning, based on j value, the root node $\in R$ sends a construction message to the directions (output ports) -1 , $+i$, and $+1$ when $j = 0$ (or, $-i$, -1 , $+i$ when $j = 1$). After that, each receiving node c performs Algorithm 1 according to the following steps. (1) c receives the construction

Table 2 Sequential construction directions.

Receiving node in	Direction
$B1^j$	$(-1)^j, (+i)^j$
$B2^j \cup B3^j$	$(-1)^j, (+1)^j$
$B4^j$ except node $-ki, j = 0$ except node $ki, j = 1$	$(-i)^j, (+1)^j$ $(-i)^j$ $(+1)^j$
$L1^j \cup L2^j$	stop
$S^j \cup D1^j$	$(-1)^j$
$D2^j$	$(+1)^j$

Algorithm 1 SequentialConstruction(c)

Require: c as receiving node

Require: Table 2

Require: j for rotation

```

1: if  $c \in \{L1^j \cup L2^j\}$  then
2:   Stop
3: end if
4: if  $c \in B1^j$  then
5:   goto  $(-1)^j$  and  $(+i)^j$  direction
6: end if
7: if  $c \in \{B2^j \cup B3^j\}$  then
8:   goto  $(-1)^j$  and  $(+1)^j$  direction
9: end if
10: if  $c \in B4^j$  and  $(c \notin -ki$  and  $j = 0)$  or  $(c \notin +k$  and  $j = 1)$  then
11:   goto  $(-i)^j$  and  $(+1)^j$  direction
12: end if
13: if  $c \in B4^j$  and  $c \in -ki$  and  $j = 0$  then
14:   goto  $(-i)^j$  direction
15: end if
16: if  $c \in B4^j$  and  $c \in +k$  and  $j = 1$  then
17:   goto  $(+1)^j$  direction
18: end if
19: if  $c \in \{S^j \cup D1^j\}$  then
20:   goto  $(-1)^j$  direction
21: end if
22: if  $c \in D2^j$  then
23:   goto  $(+1)^j$  direction
24: end if

```

message from the sender with the value of j . (2) c looks up the Table 2 to determine the direction(s) that the construction message must be forwarded. (3) based on the location of c (i.e., the subset it belongs to), c forwards the construction message to the proper direction(s) (output port(s)). Finally, (4) the algorithm stops when c is located

in either $L1^j$ or $L2^j$. Thus, the CIST based on the value of j (whether the first or second CIST) is constructed once the Algorithm 1 stops.

Algorithm 1 takes $O(n)$ time complexity due to the table lookup to determine the proper forwarding direction. And, it takes $O(1)$ communication complexity in each node (i.e., a total of $O(d)$, where d is the depth of the tree).

Example 1. Fig. 2 and Fig. 3 illustrate the first and second Completely Independent Spanning Trees in Gaussian network generated by $\alpha = 4 + 5i$.

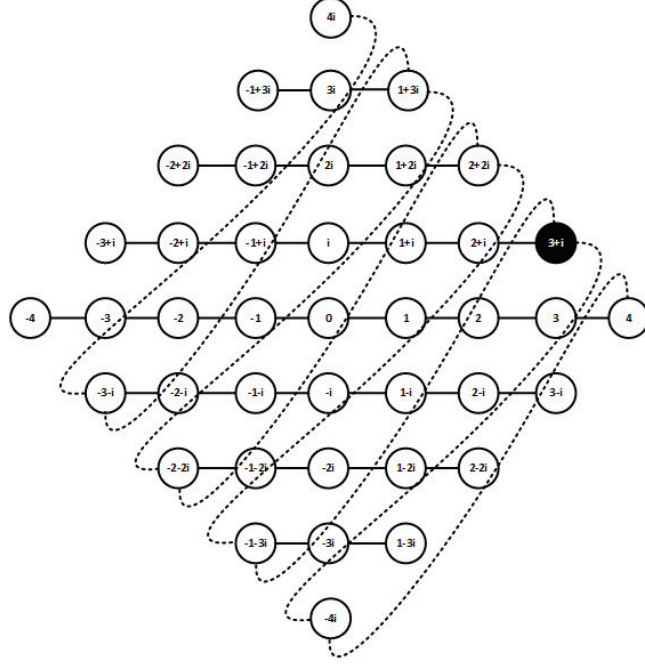


Fig. 2 First CIST in Gaussian network with $\alpha = 4 + 5i$.

5.2 Parallel Construction

In parallel construction of CISTs in dense Gaussian networks, it is assumed that each node has information about the network (that is, α) and Table 3. Initially, the information can be distributed among the nodes in the network using the broadcasting algorithm in [14], which takes $O(k)$ where k is the diameter of the network. The parallel construction algorithm is performed simultaneously by each node in the network. To construct the $CIST_1$, let $j = 0$, then each node uses Table 3 to determine its parent and child nodes based on its location in the subset of the network. The current node connects to its parent and child nodes once they are determined. In the end, we obtain the CISTs in dense Gaussian networks. To obtain $CIST_2$, we apply the same

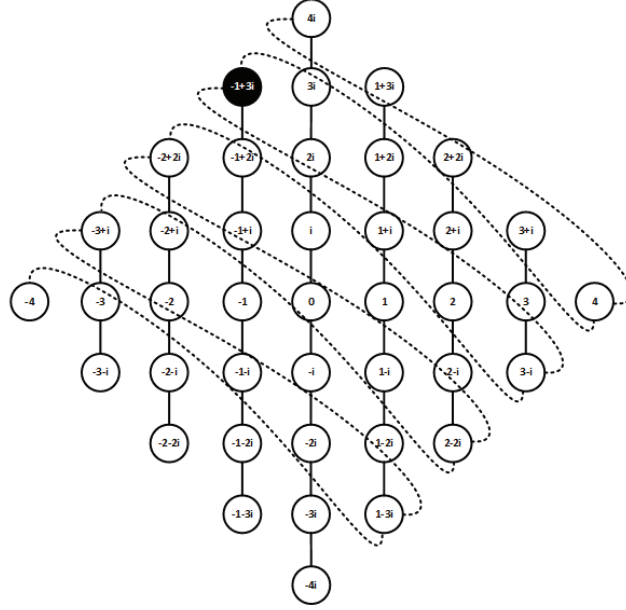


Fig. 3 Second CIST in Gaussian network with $\alpha = 4 + 5i$.

construction technique, but when $j = 1$. Algorithm 2 illustrates the construction of CISTs in dense Gaussian networks.

Table 3 Parent and child nodes for the CISTs.

Node c in subset	Parent	Child
R^j	—	$(+1)^j, (+i)^j, (-1)^j$
$B1^j$	$(+1)^j$	$(-1)^j, (+i)^j$
$B2^j$	$(+i)^j$	$(-1)^j, (+1)^j$
$B3^j$	$(-i)^j$	$(-1)^j, (+1)^j$
$B4^j$	$(-1)^j$	$(-i)^j, (+1)^j$ node $-ki$ has only $(-i)^j, j = 0$ node ki has only $(+1)^j, j = 1$
S^j	$(+i)^j$	$(-1)^j$
$L1^j$	$(+1)^j$	—
$L2^j$	$(-1)^j$	—
$D1^j$	$(+1)^j$	$(-1)^j$
$D2^j$	$(-1)^j$	$(+1)^j$

Since the total number of subsets is 10, then the time complexity of Algorithm 2 is constant $O(10)$, where 10 is the total number of subsets of the network. This is because node $c = x_c + y_c\rho$ compares its x_c and y_c with the conditions of each subset to determine its location. In addition, the communication complexity is $O(1)$ since all nodes work in parallel. The general complexity of communication takes $O(k)$ due to the initial step mentioned above.

Algorithm 2 ParallelConstruction(j, c , Table 3)

Require: j is the spanning tree number (number of rotations)

Require: c is the current node applying this algorithm

Require: Table 3

- 1: Rotate the Table 3 based on the value of j
 - 2: Lookup Table 3 and determine the subset that c belongs to and then get the parent and child nodes of c
 - 3: Connect the links of the parent and child nodes that are incident to c with c
-

6 Routing using CISTs

In this section, we describe how the routing is performed using the CISTs in Gaussian networks. Table 4 illustrates the path to the destination node D when D is in a certain subset. To determine the D 's subset, we compare the x and y of D with the subsets condition described in Section 4. The corresponding path P from R to D can be obtained once the subset is determined.

Usually, routing a message starts at the root node R to a specific destination node D . Based on the CISTs construction discussed in Section 5, dividing the network into subsets helps in designing the routing method. To route a message m from R to D using the first CIST, we perform the following steps. At the beginning, in Algorithm 3, the R looks up the Table 4 to obtain the path based on the subset where D belongs to. A path P is a list of elements consisting of pair of direction (d) and steps (s), i.e. $[(d, s), \dots, (d, s)]$, which is described in Table 4 as a sequence of d^s . The direction d determines which edge can be used to forward the message to a neighboring node, while the steps s indicate the number of hops required to traverse in the specified direction. After that, R sends D , P , and m to its neighboring node using direction d with the required $s - 1$ steps, since this step is counted. Then, in Algorithm 4, each receiving neighboring node continues sending m on the same direction d and counting down s until $s = 0$. When $s = 0$, the receiving node gets the next (d, s) from P when P is not empty. The process continues until P is empty and $s = 0$, which means that the message has been delivered to D . The same method can be applied to route the message in the second CIST, except that we need to perform the rotation as discussed in Lemma 5.

Algorithm 3 InitRoute(D)

Require: $D = x + yi$ as a destination node

- 1: Determine D 's subset using Table 4 and get the path P
 - 2: $(d, s) = P.pop()$
 - 3: Send m through d using Rout($D, P, s - 1, m$)
-

As for the theoretical analysis, the time complexity of Algorithm 3 is $O(1)$ since we have 9 subsets' conditions to be tested, and its communication complexity is $O(1)$ since we have 1 send. The time and communication complexities of Algorithm 4 is $O(1)$.

Algorithm 4 Route(D, P, s, m)

Require: $D = x + yi$ as a destination node

Require: P is the path to D

Require: s is the number of steps to be counted

Require: m is the message to be delivered

```
1: if  $s = 0$  &  $P \neq \phi$  then  
2:    $(d, s) = P.pop()$   
3:   Send  $m$  through  $d$  using Rout( $D, P, s - 1, m$ )  
4: else if  $s = 0$  then  
5:   Consume  $m$   
6: else  
7:   Send  $m$  through  $d$  using Rout( $D, P, s - 1, m$ )  
8: end if
```

Table 4 Routing paths using CISTs.

Node in subset	Path (steps)
$B1$	$((+i)(-1))^{ y-y_r }$
$B2$	$((+1)(-i))^{ y-y_r }$
$B3$	$(+i)((-1)(+i))^{k- y - y_r }$
$B4$	$(+1)((-i)(+1))^{k- y }$
S	$((+1)(-i))^{ y-y_r }$
$L1 \cup D1$ $y \geq y_r$	$((+i)(-1))^{ y-y_r } (-1)^{k-y-x}$
$L1 \cup D1$ $0 \leq y < y_r$	$((+1)(-i))^{ y-y_r } (-1)^{k-y-x}$
$L2 \cup D2$ $-x_r \leq y < 0$	$(+i)((-1)(+i))^{k- y - y_r } (+1)^{k- y +x}$
$L2 \cup D2$ $-k \leq y < -x_r$	$(+1)((-i)(+1))^{ y_r - x -1} (+1)^{k- y +x}$

The height in [15] is approximately $n/2$ where n is $N(\alpha) = (2k^2 + 2k + 1)$, whereas the height of our constructed trees is $3k - 1$ where k is the diameter of the network. Table 5 shows the difference in the tree depths of the construction algorithm in [15] and the proposed one. It is obvious that the depth of the proposed construction algorithm is much lower than the one in [15] when network size is increased with at least 50% of improvement on larger network sizes. That is, the lower depth we have the less time we get to deliver a message from the root node to the farthest node in the tree. This affects positively in decreasing the average distance of the network.

Theorem 8 Table 4 presents the shortest path from the root node $r = x_r + y_r i \in R$ to any destination node $u = x + yi$.

Proof Each elementary move in the network corresponds to a unit step in the Gaussian integer lattice. That is, $(+1) = (1, 0)$, $(-1) = (-1, 0)$, $(+i) = (0, 1)$, $(-i) = (0, -1)$, and linearity

Table 5 Depth comparison

Network Size	Construction in [15]	Proposed
$k = 2$	2	2
$k = 3$	6	5
$k = 4$	12	8
$k = 5$	20	11
$k = 6$	30	14
$k = 7$	42	17
$k = 8$	56	20
$k = 9$	72	23

of displacement under concatenation of moves. For a path from r to u the required net displacement is $(\Delta x, \Delta y) = (x - x_r, y - y_r)$. The Manhattan distance $d_{\min}(r, u) = |\Delta x| + |\Delta y|$ is a lower bound on any path length since each unit move changes $|\Delta x| + |\Delta y|$ by at most 1. The expression of the path steps in Table 4 represents the route from r to u , where the Gaussian integer is the direction of movement and the power is the number of steps (moves) required. That is, $(+i)^m$ means move m steps along the direction $+i$. We verify representative cases; the remaining rows follow the same symmetry argument.

Case B1.

The path from $r \in R$ to $u \in B1$ is $((+i)(-1))^m$ where $m = |y - y_r|$ is the vertical level difference between the node u and r . Each block $((+i)(-1))$ contributes a vector $(-1, 1)$, which represents two consecutive moves that are $(0, 1)$ then $(-1, 0)$. That is, $(0, 1)$ is a vertical move from set R or $B1$ to the sets $B3$ and S ; and, $(-1, 0)$ is a horizontal move to a node in the set $B1$ coming from the set $B3$, both moves use wraparound links. The m is the displacement of the movements between r and u that represents the number of repetitions of the block (moves) required to reach node $u \in B1$. For any node u in the set $B1$ (under partition symmetries), the displacement r to u satisfies $(\Delta x, \Delta y) = (-m, m)$, and therefore the path reaches u . Thus, the path length is $2m$, which is equal to $|\Delta x| + |\Delta y|$ and is a proper path from r to u .

Case B2 and S.

This case is similar to the case $B1$, except that the moves start with $(+1)$ then $(-i)$. That is, each block $((+1)(-i))$ contributes a vector $(1, -1)$, which $(1, 0)$ is a horizontal move from sets R or $B2$ to the set $B4$ and $(0, -1)$ is a move from the set $B4$ to the set $B2$. We can apply a similar argument to that in case $B1$ to show that the path from $r \in R$ to $u \in B2 \cup S$ is a proper path and has a length equal to $2m$ where $m = |y - y_r|$.

Case B3 and B4.

This case is similar to the cases in the above and we can apply the same argument, except that it goes one step through direction $(+i)$ to move to the set $B3$ then w steps of block $((-1)(+i))^w$ to move back and forth between the sets $B3$ and $B1$ where $w = k - |y| - |y_r|$ is the vertical level difference between the nodes u and r . The displacement is $(+i) + w(-1, 1) = (-w, w + 1)$, and this equals $(\Delta x, \Delta y)$ for nodes in the set $B3$. The number of moves is $1 + 2w = |\Delta x| + |\Delta y|$, so the path is proper. As for $B4$ we follow the same argument, but for one more through $(+1)$ and w moves for block $((-i)(+1))$ where $w = k - |y|$.

Case $L1 \cup D1$.

In this case, the destination node u is located in either the sets $L1$ or $D1$. When $y \geq y_r$, the moves are divided into two parts. The first part, the moves are from $r \in R$ to nodes in the set $B1$ using block $((+i)(-1))$ as in case $B1$ until the vertical level of u is reached; the same argument of case $B1$ can be used. Then (the second part), the moves are through the direction (-1) by $k - y - x$ steps, which is the required number of moves from any node $v = v_x + v_y i \in B1$ to node $u \in L1 \cup D1$ on the same vertical level. That is, when $v \in B1$ then $v_x + v_y = k$. That is, the distance to u from v can be at most $2k$. Thus, $k - y - x$ is the required number of steps to move from v to u . On the other hand, when $0 \geq y < y_r$ we follow the same argument, except that for block $((+1)(-i))$ where it moves from $r \in R$ to $u \in B4$.

Case $L2 \cup D2$.

The same argument as in case $L1 \cup D1$ can be followed to prove this case.

The above arguments confirm that the paths are proper. The following shows that the paths in Table 4 are shortest. Let $T = (V, E)$ be any CIST in Gaussian network. By definition, a tree is an acyclic and connected graph. For any two vertices $u, v \in V$, since T is connected, there exists at least one path between u and v . Because T is acyclic, this path is unique. If there were another shorter path between u and v , it would be distinct from the unique path, creating a contradiction. Therefore, the unique path between any two vertices in a tree must be the shortest possible path.

Therefore, the constructed paths are valid and attain the shortest possible length. $\square$

7 Experimental Results

This section details the experiments conducted to evaluate the performance of the proposed method and it is divided into two subsections. In Subsection 7.1 we discuss the experimental setup where the simulation result is discussed in Subsection 7.2.

7.1 Experimental Setup

The simulation is implemented using the Python package NetworkX [33], which provides tools for developing, modeling, and visualizing networks as graphs, leveraging standard graph theory functions and algorithms.

The experiments were conducted on a MacBook Pro equipped with an Intel Core i9 processor running at 2.4 GHz (8 cores, boosted to 5GHz) and 32 GB of RAM. All simulation programs were implemented in Python using the NetworkX library for graph construction and analysis. This hardware and software environment was sufficient to efficiently construct the Gaussian networks, generate the CISTs, and perform the routing and fault-tolerance experiments presented in this paper.

For this study, we assume that all network links are full-duplex, meaning that nodes can send and receive data simultaneously. The construction algorithms were tested under three scenarios: (1) no faulty nodes in the network, (2) one faulty node present, and (3) two faulty nodes present. In cases involving faulty nodes, we applied a brute-force approach, testing all possible locations of the faulty node(s). The network sizes tested include: $3 + 4i$, $4 + 5i$, $5 + 6i$, $6 + 7i$, $7 + 8i$, $8 + 9i$, $9 + 10i$, $10 + 11i$, $11 + 12i$, and $12 + 13i$.

In case of the absence of a faulty node, we calculate the shortest-path distance from a root node, say r , to each node u in the first CIST (CIST1). The same calculation is performed on the second CIST (CIST2). After that, for each node u in the network, we compare the path length from r to u in CIST1 with the path length from r to u in CIST2 to obtain the maximum distance between CISTs for the paths from r to each node u . In the end, since we have the maximum distances to each node u in the network, the average of the maximum distance is computed.

In case of the presence of a faulty node, a similar method is used as in case of the absence of a faulty node, except that the node failure is counted in all possible locations. The brute-force approach is applied to measure the average maximum steps to deliver a message from the root node, say r , to all other nodes in the network as follows. Once the network is generated and the CISTs are constructed, a node is selected as a failure node in the network. Using CIST1, the path lengths (distance) from r to each node u in CIST1 are calculated. After that, the faulty node is returned with its links as an active node and another node is selected as a faulty node. The path lengths from r to each node u are again calculated, where the values of each path length are updated to the maximum distances compared with the previous computed distances. This process continues until all possible faulty node locations are counted. The same method is applied using CIST2 with the distance values updated to the maximum. Finally, the average of the maximum distances is computed.

In case of the presence of two faulty nodes, a similar approach to the case of one faulty node is followed, except that all possible locations of two faulty nodes are considered.

7.2 Experimental Outcomes

Table 5 compares the tree depths of the constructed CISTs across different network sizes, ranging from $k = 2$ ($\alpha = 2 + 3i$) to $k = 9$ ($\alpha = 9 + 10i$). The proposed method consistently results in shallower trees compared to the method in [15], which contributes significantly to the improved network performance.

The experimental results demonstrate the effectiveness of the proposed method in constructing CISTs for networks with varying sizes and fault conditions. The method consistently outperforms previous approaches, particularly in terms of reducing the depth of the constructed trees and minimizing the number of steps required for message delivery. These improvements are evident in all tested cases, including scenarios with faulty nodes. The reduction in tree depth plays an important role in improving overall network performance, and the demonstrated improvement of at least 26.56% over previous work further validates the robustness of our approach. This makes the proposed method a viable solution for optimizing communication in fault-tolerant networks.

Table 6 Average maximum number of steps to route a message in all trees.

α	$3 + 4i$	$4 + 5i$	$5 + 6i$	$6 + 7i$	$7 + 8i$	$8 + 9i$	$9 + 10i$	$10 + 11i$	$11 + 12i$	$12 + 13i$
No Faulty	2.40	3.32	3.80	4.69	5.27	6.12	6.73	7.57	8.22	9.04
1 Faulty	3.72	5.17	6.84	8.20	10.00	11.31	13.19	14.46	16.39	17.63
2 Faulty	3.26	4.74	6.37	7.77	9.52	10.87	12.70	14.00	15.89	17.16

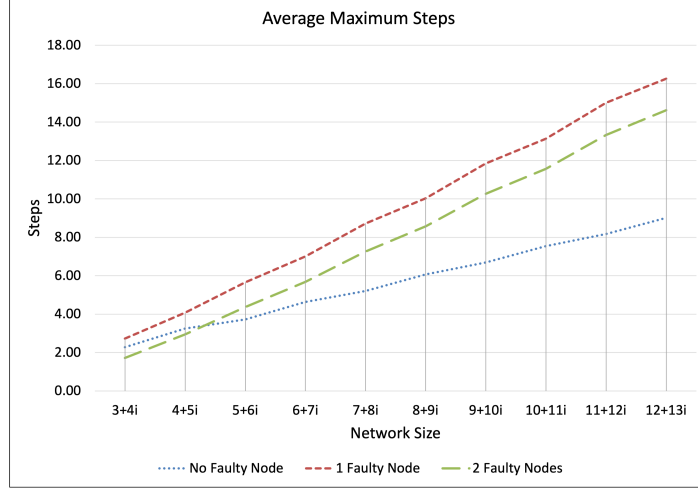


Fig. 4 Average Maximum Steps.

Table 7 No Faulty Nodes Comparisons.

Network Size	3 + 4i	4 + 5i	5 + 6i	6 + 7i	7 + 8i	8 + 9i	9 + 10i	10 + 11i	11 + 12i	12 + 13i
Proposed	2.40	3.32	3.80	4.69	5.27	6.12	6.73	7.57	8.22	9.04
Referenced [15]	4.83	7.38	10.83	14.70	19.50	24.71	30.84	37.39	44.85	52.73
Improvement %	50.31	55.05	64.88	68.07	73.00	75.24	78.16	79.94	81.67	82.85

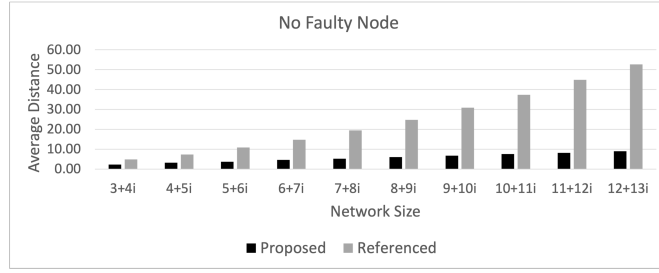


Fig. 5 No Faulty Nodes Comparisons.

Table 8 One Faulty Node Comparisons.

Network Size	3 + 4i	4 + 5i	5 + 6i	6 + 7i	7 + 8i	8 + 9i	9 + 10i	10 + 11i	11 + 12i	12 + 13i
Proposed	3.72	5.17	6.84	8.20	10.00	11.31	13.19	14.46	16.39	17.63
Referenced [15]	5.07	7.69	11.29	15.23	20.16	25.47	31.69	38.34	45.86	53.87
Improvement %	26.56	32.73	39.43	46.13	50.38	55.58	58.37	62.28	64.26	67.27

8 Conclusion

In this research, we addressed the challenge of constructing Completely Independent Spanning Trees CISTs within Gaussian networks. By partitioning the network into ten subsets and exploiting its inherent symmetry, we effectively uncovered two CISTs. The

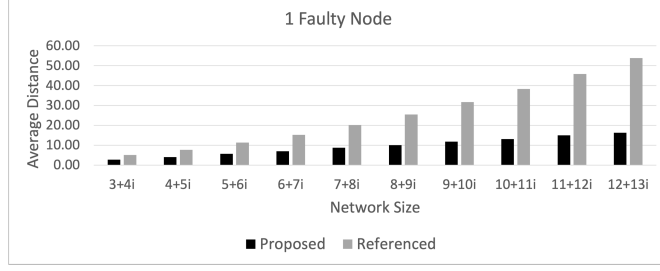


Fig. 6 One Faulty Node Comparisons.

Table 9 Two Faulty Nodes Comparisons.

Network Size	3 + 4i	4 + 5i	5 + 6i	6 + 7i	7 + 8i	8 + 9i	9 + 10i	10 + 11i	11 + 12i	12 + 13i
Proposed	3.26	4.74	6.37	7.77	9.52	10.87	12.70	14.00	15.89	17.16
Referenced [15]	4.92	7.53	11.07	14.93	19.79	25.01	31.15	37.69	45.11	52.99
Improvement %	33.64	36.99	42.44	47.94	51.88	56.55	59.23	62.85	64.77	67.62

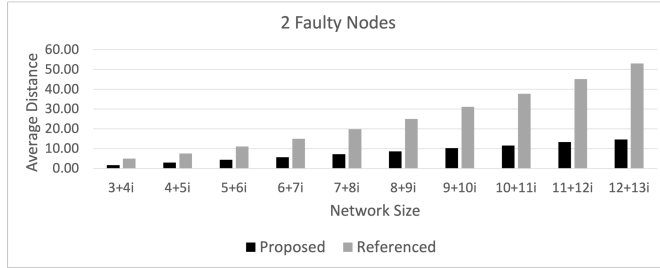


Fig. 7 Two Faulty Nodes Comparisons.

first tree was constructed from the network's subsets, while the second was generated by rotating the first tree in a counterclockwise direction. This approach allowed for the systematic construction of CISTs while maintaining the structural properties of the network.

We introduced both sequential and parallel algorithms for constructing these trees, along with a routing algorithm tailored to the resulting network. A key result of our work is the consistent depth of all constructed trees, which we proved to be $3k - 1$. In addition to this, we analyzed the time and communication complexities of the proposed algorithms, demonstrating their efficiency in practice.

In comparison to the previous work by [15], our method offers significant improvements. Notably, our construction method results in trees with shallower depths and reduced average distances between nodes, which directly enhance network performance. This improvement is critical for optimizing communication in Gaussian networks, especially in fault-tolerant environments, making our approach a more effective and scalable solution for CIST construction.

References

- [1] Dally, W., Seitz, C.: The torus routing chip. *Distributed Computing* **1** (1986) <https://doi.org/10.1007/BF01660031>
- [2] Ajima, Y., Kawashima, T., Okamoto, T., Shida, N., Hirai, K., Shimizu, T., Hiramoto, S., Ikeda, Y., Yoshikawa, T., Uchida, K., Inoue, T.: The tofu interconnect d. In: 2018 IEEE International Conference on Cluster Computing (CLUSTER), pp. 646–654 (2018). <https://doi.org/10.1109/CLUSTER.2018.00090>
- [3] Kim, J., Dally, W., Scott, S., Abts, D.: Technology-driven, highly-scalable dragonfly topology, vol. 36, pp. 77–88 (2008). <https://doi.org/10.1109/ISCA.2008.19>
- [4] Flahive, M., Bose, B.: The topology of gaussian and eisenstein-jacobi interconnection networks. *IEEE Transactions on Parallel and Distributed Systems* **21**(8), 1132–1142 (2010) <https://doi.org/10.1109/TPDS.2009.132>
- [5] Martínez, C., Vallejo, E., Beivide, R., Izu, C., Moretó, M.: Dense gaussian networks: Suitable topologies for on-chip multiprocessors. *International Journal of Parallel Programming* **34**, 193–211 (2006) <https://doi.org/10.1007/s10766-006-0014-1>
- [6] Guo, C., Lu, G., Li, D., Wu, H., Zhang, X., Shi, Y., Tian, C., Zhang, Y., Lu, S.: Bcube: a high performance, server-centric network architecture for modular data centers. In: *Proceedings of the ACM SIGCOMM 2009 Conference on Data Communication*. SIGCOMM '09, pp. 63–74. Association for Computing Machinery, New York, NY, USA (2009). <https://doi.org/10.1145/1592568.1592577>
- [7] Bose, B., Broeg, B., Kwon, Y., Ashir, Y.: Lee distance and topological properties of k-ary n-cubes. *IEEE Transactions on Computers* **44**(8), 1021–1030 (1995) <https://doi.org/10.1109/12.403718>
- [8] Hayes, J.P., Mudge, T.: Hypercube supercomputers. *Proceedings of the IEEE* **77**(12), 1829–1841 (1989) <https://doi.org/10.1109/5.48826>
- [9] Adiga, N.R., Almasi, G., Almasi, G.S., Aridor, Y., Barik, R., Beece, D., Bellofatto, R., Bhanot, G., Bickford, R., Blumrich, M., Bright, A.A., Brunheroto, J., Cascaval, C., Castanos, J., Chan, W., Ceze, L., Coteus, P., Chatterjee, S., Chen, D., Yates, K.: An overview of the bluegene/l supercomputer, pp. 60–60 (2002). <https://doi.org/10.1109/SC.2002.10017>
- [10] Shimizu, T.: Supercomputer fugaku: Co-designed with application developers/researchers. In: 2020 IEEE Asian Solid-State Circuits Conference (A-SSCC), pp. 1–4 (2020). <https://doi.org/10.1109/A-SSCC48613.2020.9336127>

- [11] TOP 500 Supercomputer Sites: TOP 500 Supercomputer Sites. <http://www.top500.org>. Accessed: August 4, 2024
- [12] Atchley, S., Zimmer, C., Lange, J., Bernholdt, D., Melesse Vergara, V., Beck, T., Brim, M., Budiardja, R., Chandrasekaran, S., Eisenbach, M., Evans, T., Ezell, M., Frontiere, N., Georgiadou, A., Glenski, J., Grete, P., Hamilton, S., Holmen, J., Huebl, A., Jacobson, D., Joubert, W., McMahon, K., Merzari, E., Moore, S., Myers, A., Nichols, S., Oral, S., Papatheodore, T., Perez, D., Rogers, D.M., Schneider, E., Vay, J.-L., Yeung, P.K.: Frontier: Exploring exascale. In: Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis. SC '23. Association for Computing Machinery, New York, NY, USA (2023). <https://doi.org/10.1145/3581784.3607089>
- [13] HPE: HPE SLINGSHOT INTERCONNECT. <https://www.hpe.com/us/en/compute/hpc/slingshot-interconnect.html>. Accessed: August 4, 2024 (2022)
- [14] Martínez, C., Beivide, R., Stafford, E., Moreto, M., Gabidulin, E.: Modeling toroidal networks with the gaussian integers. Computers, IEEE Transactions on **57**, 1046–1056 (2008) <https://doi.org/10.1109/TC.2008.57>
- [15] Pai, K.-J., Yang, J.-S., Chen, G.-Y., Chang, J.-M.: Configuring protection routing via completely independent spanning trees in dense gaussian on-chip networks. IEEE Transactions on Network Science and Engineering **9**(2), 932–946 (2022)
- [16] Dally, W.J., Towles, B.P.: Principles and Practices of Interconnection Networks. Morgan Kaufmann Publishers Inc., San Francisco, CA, USA (2004)
- [17] Alsaleh, O., Bose, B., Hamdaoui, B.: One-to-many node-disjoint paths routing in dense gaussian networks. The Computer Journal **58**(2), 173–187 (2015) <https://doi.org/10.1093/comjnl/bxt142>
- [18] Zhang, Z., Guo, Z., Yang, Y.: Efficient all-to-all broadcast in gaussian on-chip networks. IEEE Transactions on Computers **62**(10), 1959–1971 (2013) <https://doi.org/10.1109/TC.2012.126>
- [19] Touzene, A.: On all-to-all broadcast in dense gaussian network on-chip. IEEE Transactions on Parallel and Distributed Systems **26**(4), 1085–1095 (2015) <https://doi.org/10.1109/TPDS.2014.2314689>
- [20] Shamaei, A., Bose, B., Flahive, M.: Higher dimensional gaussian networks. In: 2014 IEEE International Parallel and Distributed Processing Symposium Workshops, pp. 1438–1447 (2014). <https://doi.org/10.1109/IPDPSW.2014.161>
- [21] Hussain, Z., AlBdaiwi, B., Cerny, A.: Node-independent spanning trees in gaussian networks. Journal of Parallel and Distributed Computing **109**, 324–332 (2017) <https://doi.org/10.1016/j.jpdc.2017.06.018>

- [22] Chang, Y.-H., Yang, J.-S., Chang, J.-M., Wang, Y.-L.: A fast parallel algorithm for constructing independent spanning trees on parity cubes. *Applied Mathematics and Computation* **268**, 489–495 (2015) <https://doi.org/10.1016/j.amc.2015.06.081>
- [23] Yang, J., Wu, M., Chang, J., *et al.*: A fully parallelized scheme of constructing independent spanning trees on möbius cubes. *The Journal of Supercomputing* **71**, 952–965 (2015) <https://doi.org/10.1007/s11227-014-1346-z>
- [24] Fragopoulou, P., Akl, S.G.: Edge-disjoint spanning trees on the star network with applications to fault tolerance. *IEEE Transactions on Computers* **45**(2), 174–185 (1996) <https://doi.org/10.1109/12.485370>
- [25] AlBdaiwi, B., Hussain, Z., Cerny, A., Aldred, R.: Edge-disjoint node-independent spanning trees in dense gaussian networks. *The Journal of Supercomputing* **72**(12), 4718–4736 (2016) <https://doi.org/10.1007/s11227-016-1768-x>
- [26] Hasunuma, T.: Completely independent spanning trees in the underlying graph of a line digraph. *Discrete Mathematics* **234**(1), 149–157 (2001) [https://doi.org/10.1016/S0012-365X\(00\)00377-0](https://doi.org/10.1016/S0012-365X(00)00377-0)
- [27] Hasunuma, T.: Completely independent spanning trees in maximal planar graphs. In: Goos, G., Hartmanis, J., Leeuwen, J., Kučera, L. (eds.) *Graph-Theoretic Concepts in Computer Science. WG 2002. Lecture Notes in Computer Science. Lecture Notes in Computer Science*, vol. 2573, pp. 228–241. Springer, ??? (2002). https://doi.org/10.1007/3-540-36379-3_21
- [28] Moinet, A., Darties, B., Gastineau, N., Baril, J.-L., Togni, O.: Completely independent spanning trees for enhancing the robustness in ad-hoc networks. In: 2017 IEEE 13th International Conference on Wireless and Mobile Computing, Networking and Communications (WiMob), pp. 63–70 (2017). <https://doi.org/10.1109/WiMOB.2017.8115791>
- [29] Wang, Y., Cheng, B., Qian, Y., Wang, D.: Constructing completely independent spanning trees in a family of line-graph-based data center networks. *IEEE Transactions on Computers* **71**(5), 1194–1203 (2022) <https://doi.org/10.1109/TC.2021.3077587>
- [30] Chen, G., Cheng, B., Wang, D.: Constructing completely independent spanning trees in data center network based on augmented cube. *IEEE Transactions on Parallel and Distributed Systems* **32**(3), 665–673 (2021) <https://doi.org/10.1109/TPDS.2020.3029654>
- [31] Li, X.-Y., Lin, W., Liu, X., Lin, C.-K., Pai, K.-J., Chang, J.-M.: Completely independent spanning trees on bccc data center networks with an application to fault-tolerant routing. *IEEE Transactions on Parallel and Distributed Systems* **33**(8), 1939–1952 (2022) <https://doi.org/10.1109/TPDS.2021.3133595>

- [32] Pai, K.-J., Chang, R.-S., Wu, R.-Y., Chang, J.-M.: Three completely independent spanning trees of crossed cubes with application to secure-protection routing. In: 2019 IEEE 21st International Conference on High Performance Computing and Communications; IEEE 17th International Conference on Smart City; IEEE 5th International Conference on Data Science and Systems (HPCC/SmartCity/DSS), pp. 1358–1365 (2019). <https://doi.org/10.1109/HPCC/SmartCity/DSS.2019.00189>
- [33] Hagberg, A., Swart, P.J., Schult, D.A.: Exploring network structure, dynamics, and function using networkx. Technical report, Los Alamos National Laboratory (LANL), Los Alamos, NM (United States) (2008)